1. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки. Вы обратили внимание на таблицу с информацией о зарегистрировавшихся покупателях (клиентах). Таблица имеет следующую схему:

   clients (

   client_id number, --уникальный id клиента

   client_name varchar(255), --имя клиента

   client_surname varchar(255), --фамилия клиента

   login varchar(30), --логин, который придумал клиент

   city_id number, --id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)

   age number, --возраст клиента

   reg_date date –дата регистрации на сайте);

   Считаете ли вы данный набор и смысл полей корректным? Если нет, то напишите что по вашему мнению некорректно и какие изменения внесли бы

   <u>Мой вариант</u>

   CREATE TABLE clients (

   Id INT NOT NULL client_id number PRIMERY KEY,

   first_client_name varchar(255) NOT NULL,

client_surname varchar(255) NOT NULL,

login varchar(30) NOT NULL,

city_id number,

age number,

reg_date date

FOREIGN KEY cities);

CREATE TABLE cities (

city_id number PRIMARY KEY,

city_name VARCHAR(255) NOT NULL);

Возможно нужно создать отдельно таблицу с городом и его id. Изначально был вариант как показан ниже, но меня смутило что «id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)», то есть в одной таблице будет его id, а в другой хранится его соответствующее название и id. Еще мне кажется что не хватает информации, допустим клиенту не всегда будет удобно оповещение по номеру, можно было бы его дублировать и на электронную почту.

CREATE TABLE clients (

Id INT NOT NULL client_id number PRIMERY KEY, - уникальный id клиента

first client name varchar(255) NOT NULL, - имя клиента

client surname varchar(255) NOT NULL, - фамилия клиента

login varchar(30) NOT NULL, логин, - который придумал клиент

city id number, - id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)

age number, -возраст клиента

reg date date-дата регистрации на сайте);

2. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки. Вы обратили внимание на таблицу с информацией о товарах магазина. Таблица имеет следующую схему:

items ( ); item_id number, --уникальный id товара

item_name varchar(255), --наименование товара

item_cost number –стоимость товара

Глядя на эту таблицу вы вспоминаете, что мы живём в рыночной экономике и цена, если не каждый день, то каждый месяц может меняться. К примеру, сезонные повышения цен или наоборот скидки (перед началом учебного года или новым годом). Так же вы знаете что у магазина есть отдел финансовой отчётности, в котором сотрудникам нужно строить различные отчёты (о доходах например) за разные промежутки времени, в том числе и за прошлые месяцы и даже прошлые года. Как бы вы доработали архитектуру таблицы, что бы обеспечить историческое хранение стоимости товара

Мой вариант

Отсутствует

3. Требуется проверить запрос на корректность и исправить там ошибки, если они есть. Существует учебная схема HR, содержащая таблицы: employees, departments и locations Необходимо получить все отделы, расположенные в Сеуле в которых все сотрудники, не имеющие менеджера, зарабатывают в общей сложности более 100000. Для решения задачи был написан следующий запрос:

```
SELECT DEPARTMENT_ID,
DEPARTMENT_NAME,
SUM(SALARY) TOTAL_SALARY
FROM EMPLOYEE E,
DEPARTMENTS D
WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
AND MANAGER_ID = NULL
AND LOCATION_ID = (SELECT LOCATION_ID
FROM LOCATIONS
WHERE CITY = 'SEOUL')
AND SUM(SALARY) >= 100000
GROUP BY DEPARTMENT_NAME
```

<u>Мой вариант</u>

Отсутствует

4. В некоторой комнате на пол уронили карандаш. Объясните почему вы не можете через него перепрыгнуть

<u>Мой вариант</u>

Вот здесь у меня много вариантов, допустим

1. Неизвестно какая именно комната, может она многоуровневая (для наглядности представим театр, мы на балконе, а карандаш упал на пол в партере, а вход закрыт), уронили с высоты, а чтобы перепрыгнуть мы физически не можем попасть на уровень ниже
2. Возможно у нас нога в гипсе, с такой ногой не попрыгаешь
3. При падении закатился (за шкаф, холодильник)
4. Что если он упал буквально под ноги, нет места для прыжка, только если делать шаг назад
5. Карандаш лежит у плинтуса, если и прыгать то прямиком в стену
6. Есть еще какие-то внешние факторы, которые не позволили карандашу остаться лежать на полу, ну та же собака подбежала и утащила